
Searching

- ❑ Linear search
- ❑ Self-organized list
- ❑ Binary search
- ❑ List verification

Overview

- ▶ Look for information of interest
- ▶ Information can be in lots of forms
 - ▶ Text
 - ▶ Number
 - ▶ Signals
 - ▶ Image, video
- ▶ Strategies and techniques:
 - ▶ Linear
 - ▶ Sentinel
 - ▶ Binary
 - ▶ Hierarchical
 - ▶ Pattern matching
 - ▶ ...



Linear search

Linear (sequential) search

- ▶ Sequentially visit all the elements in a list or array
- ▶ Compare the given key with each element
 - ▶ If the element is found, return its index or pointer
 - ▶ Return -1 or NULL otherwise
- ▶ The list or array is unorganized (elements in any order)

Example

```
int search(char* str, int n, char v) {  
    int t;  
    for (t = 0; t < n && v != str[t]; t++);  
  
    return t < n ? t : -1;  
}
```

```
char* str = "asdf";  
int index = search(str, 4, 's');  
printf("%d", index);
```

Sentinel

- ▶ Note that each iteration requires two conditions to be checked and one statement to be executed
- ▶ We can avoid checking for the end of the array on every iteration by inserting the target as an extra “sentinel” element at the end of the array
- ➔ No more need to check for end-of-array condition

Example

```
int search(char* str, int n, char v) {  
    str[n] = v;  
  
    int t;  
    for (t = 0; v != str[t]; t++);  
  
    str[n] = 0;  
  
    return t < n ? t : -1;  
}
```

Self organizing list

Idea

- ▶ List that reorders its elements based on some heuristic
 - ▶ Improve overall average access time
 - ▶ Frequently accessed elements are closed to the head
- ▶ Approaches
 - ▶ Move to front method
 - ▶ Transpose method
 - ▶ Count method
- ▶ Applications
 - ▶ Language processing
 - ▶ Caching algorithms

Move-to-front method

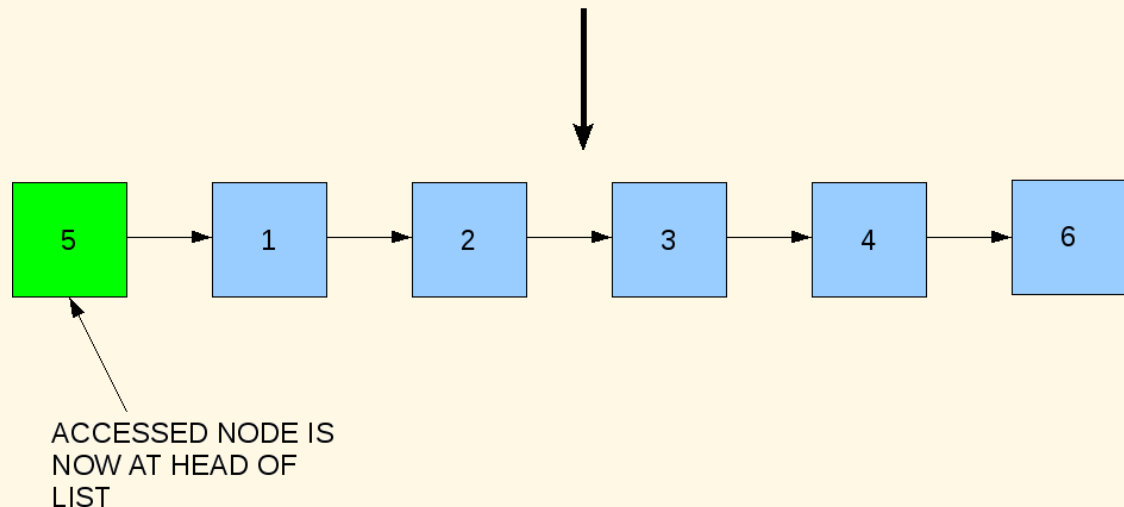
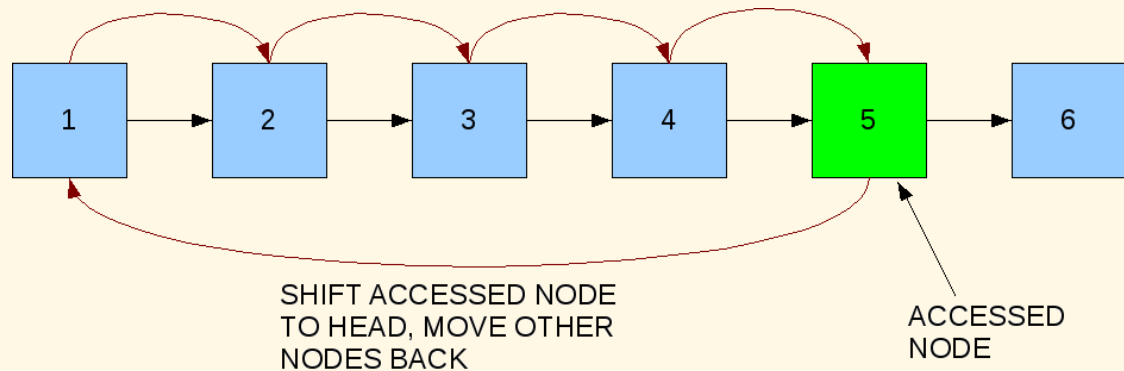
- ▶ Any element searched/accessed is moved to the front

- ▶ Pros

- ▶ No more memory needed
 - ▶ Simple

- ▶ Cons

- ▶ May over-reward

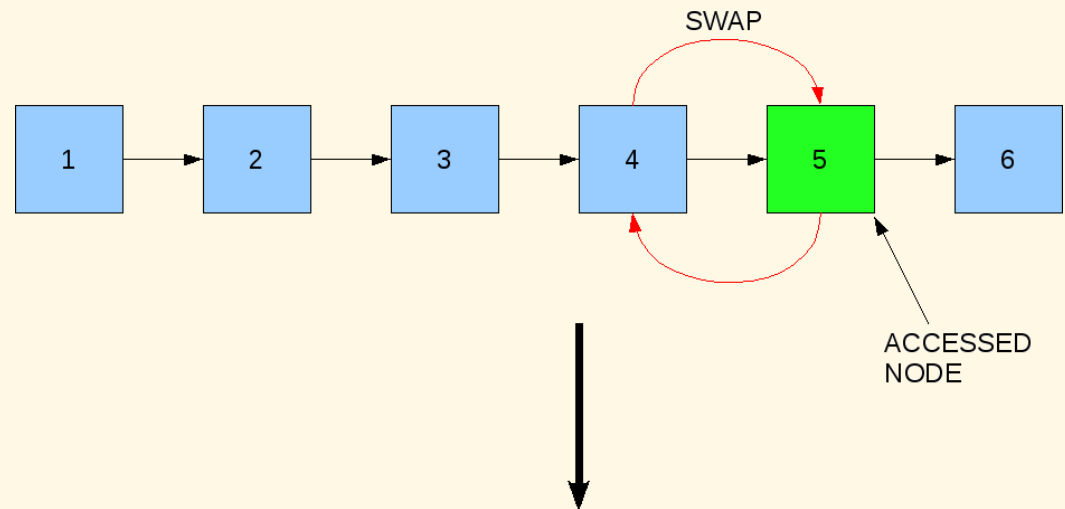


Transpose method

- ▶ Searched/accessed element is swapped with the next one

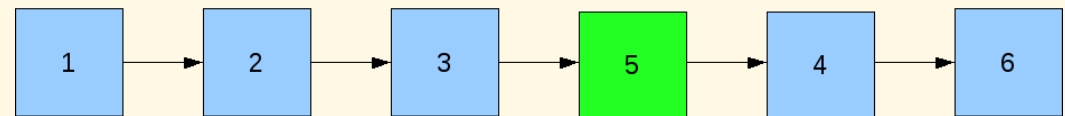
- ▶ Pros

- ▶ No more memory needed
- ▶ Simple



- ▶ Cons

- ▶ Slow convergence



Count method

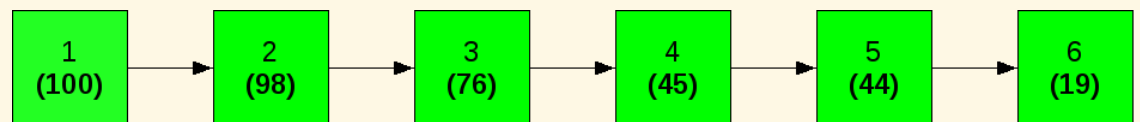
- ▶ The nodes are arranged based on the number of times they are searched for

- ▶ Pros

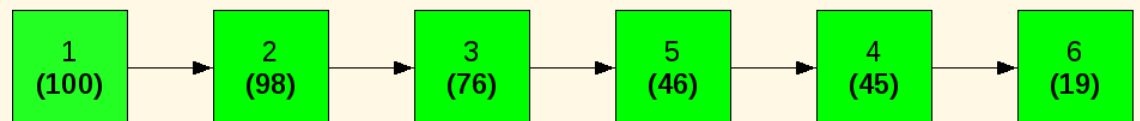
- ▶ More exact

- ▶ Cons

- ▶ Memory for counters
 - ▶ Not adaptive when access pattern changes

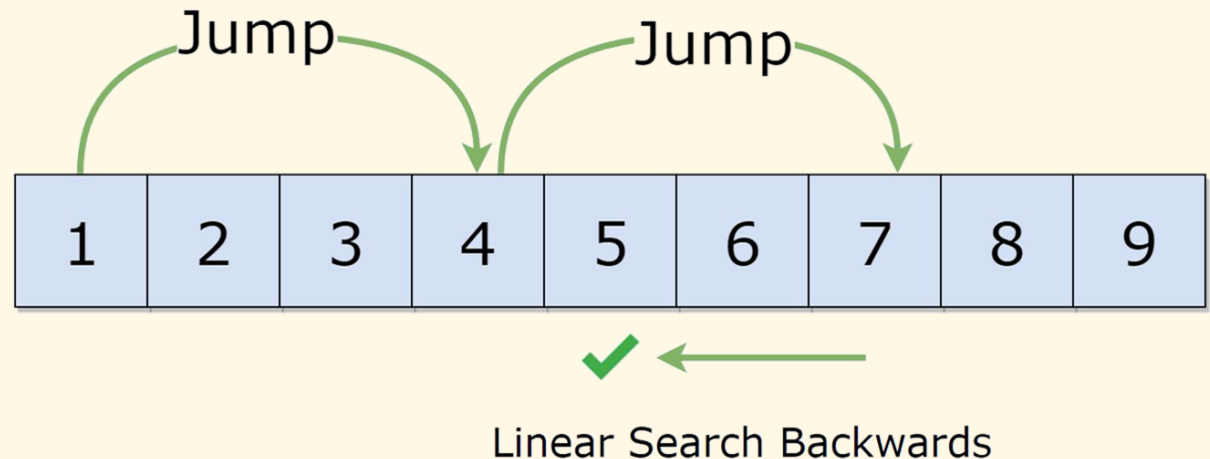


If node 5 is accessed three times its count will become 46 and it will be moved to the front



Jump search

Principle



- ▶ When data is sorted
 - ▶ Jump multiple steps (m) ahead each time to quickly narrow down the range of search
 - ▶ Linear search once the range is found
- ▶ Worst number of iterations: $n/m + m$
 - ▶ Minimal when $m = \sqrt{n}$

Implementation

```
int jump_search(int a[], int n, int v) {
    int i, j = 0, m = (int)sqrt(n);

    for (i = 0; i < n && a[i] <= v; j = i, i += m)
        if (a[i] == v) return i;

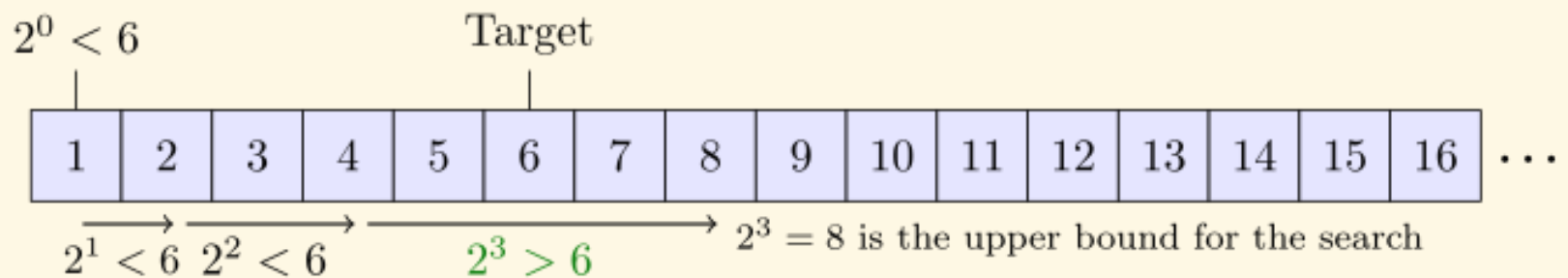
    if (i >= n) {
        i = n - 1;
        if (a[i] < v) return -1;
    } else i--;

    for (; i >= j; i--)
        if (a[i] == v) return i;

    return -1;
}
```

Further notes

- ▶ Recursively apply jump search for found ranges
- ▶ Jump search with exponentially increasing size
→ Exponential search



- ▶ Better estimation for jump targets
→ Interpolation search:

$$p = i_{start} + \frac{v - a_{start}}{a_{end} - a_{start}} (i_{end} - i_{start})$$

(not applicable for non-scalar data types)

Exercise

- ▶ Implement a jump search such that when going backward, jump search is also used
 - ▶ Note that, this will make the search recursive, and it may go forward then backward multiple times
- ▶ Implement a jump search with exponentially increasing size for double-valued arrays.

Binary search

Principle

1	3	5	6	10	11	14	25	26	40	41	78
---	---	---	---	----	----	----	----	----	----	----	----

- ▶ Similar to the way we look up words in a dictionary
 - ▶ Require the array/listed to be sorted
- ▶ Divide-and-conquer technique
 - ▶ The key is compared with the element in the middle of the list
 - ▶ If the key is smaller, restrict the search to the first half of the list
 - ▶ Otherwise, search the second half of the list
- ▶ Extremely good in reducing the number of condition checking times

Illustration

- ▶ Search key = 78

1	3	5	6	10	11	14	25	26	40	41	78
---	---	---	---	----	----	----	----	----	----	----	----

1	3	5	6	10	11	14	25	26	40	41	78
---	---	---	---	----	----	----	----	----	----	----	----

$11 < 78$

14	25	26	40	41	78
----	----	----	----	----	----

$26 < 78$

40	41	78
----	----	----

$41 < 78$

78

$78 = 78$

- ▶ Only 4 iterations needed to find

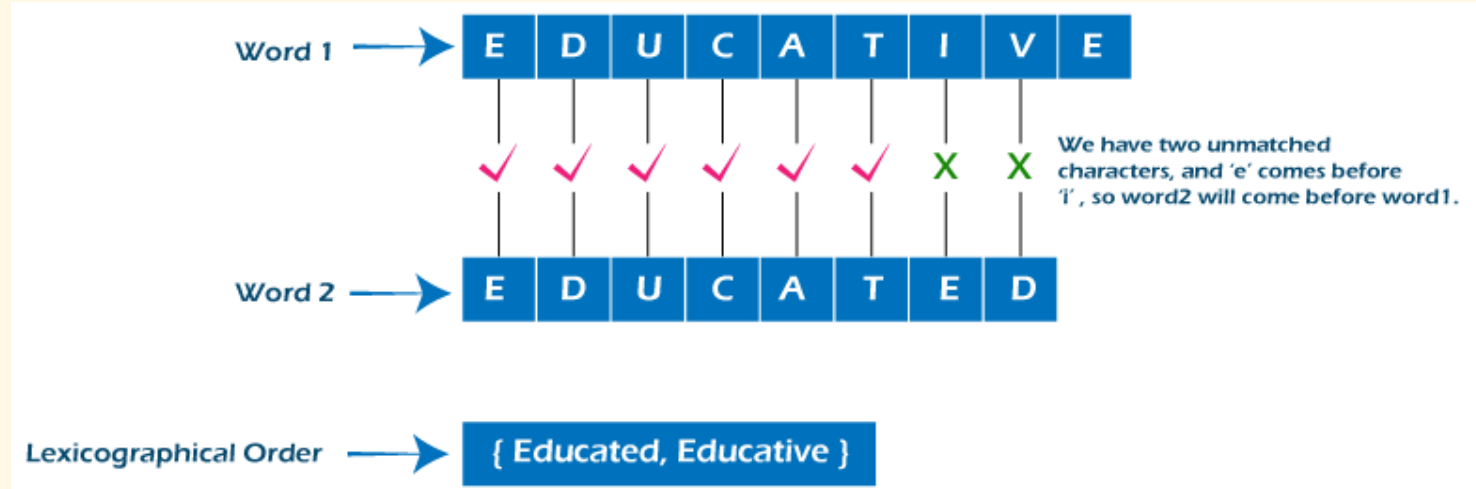
Implementation (iterative)

```
int search(int list[], int key, int lo, int hi) {  
    int mid;  
  
    while (lo <= hi) {  
        mid = (lo + hi) / 2;  
  
        if (list[mid] < key)  
            lo = mid + 1;  
        else if (list[mid] > key)  
            hi = mid - 1;  
        else return mid;  
    }  
  
    return -1;  
}
```

Implementation (recursive)

```
int search(int list[], int key, int lo, int hi) {  
    if (lo > hi) return -1;  
  
    int mid = (lo + hi) / 2;  
  
    if (list[mid] < key)  
        return search(list, key, mid + 1, hi);  
  
    if (list[mid] > key)  
        return search(list, key, lo, mid - 1);  
  
    return mid;  
}
```

String comparison with lexicographic order



- ▶ `strcmp(s1, s2)` function, returns
 - ▶ 1 if s1 is “greater” than s2
 - ▶ 0 if s1 is identical to s2
 - ▶ -1 if s1 is “lower” than s2
- ▶ Attention
 - ▶ Character comparison is based on ASCII code
 - ▶ 'a' < 'd', 'B' < 'M'
 - ▶ "adding" < "addition"

Big-O of search algorithms

- ▶ Use n as number of elements in the array/list
- ▶ Estimate the number of iterations needed to find the key
 - ▶ Average value (most important)
 - ▶ Worst case
 - ▶ Best case
- ▶ Sequential search: $O(n)$
- ▶ Jump search: $O(\sqrt{n})$
- ▶ Binary search: $O(\log n)$

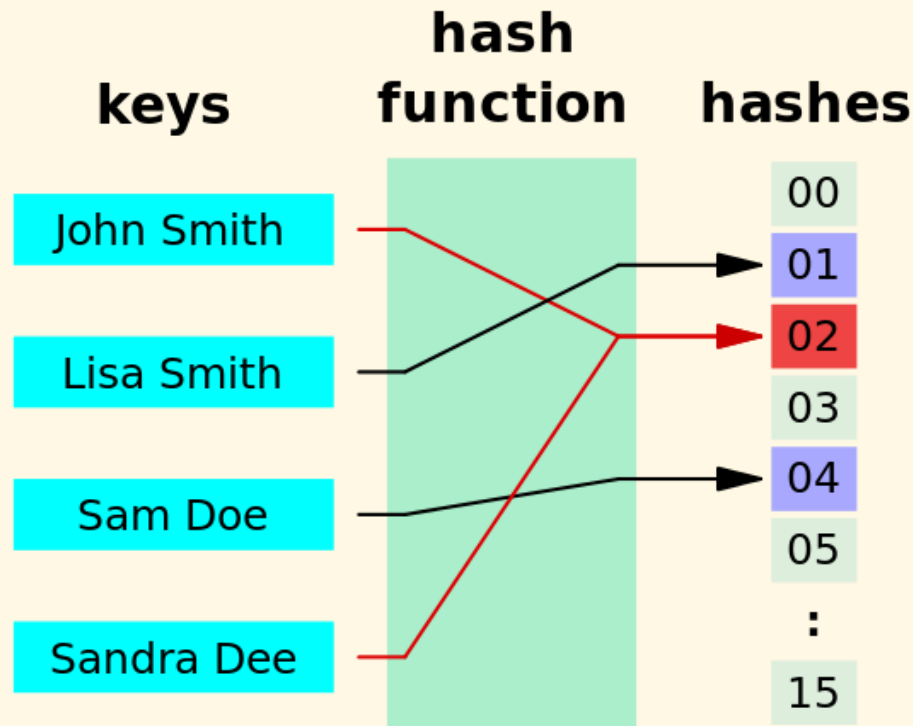
Exercises

- ▶ Implement a binary search for string-valued arrays.
- ▶ Do a real test to compare sequential, jump, binary search algorithms.

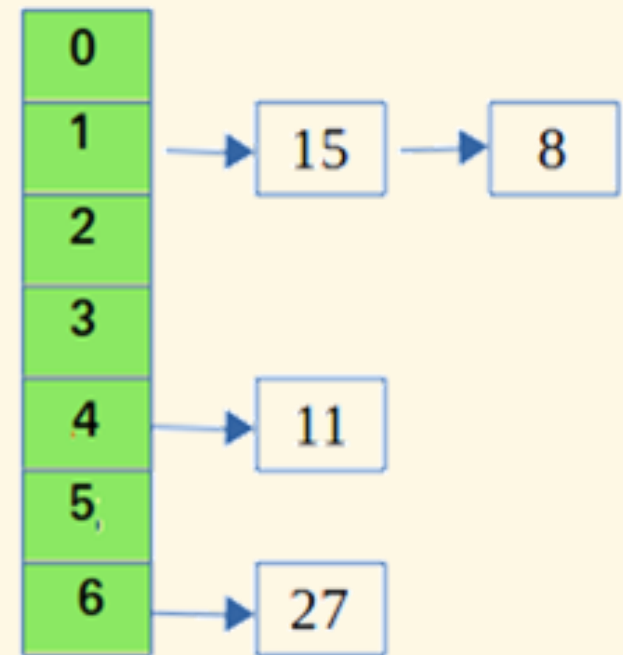
Hash function

Overview

- ▶ Arrange the data into groups with deterministic keys



Chaining



- ▶ To be revisited later!

List verification

Overview

- ▶ Compare two lists to verify that they are identical or identify the discrepancies
- ▶ Example: international revenue service (e.g., employee vs. employer)
- ▶ Complexity
 - ▶ Unordered lists: $O(mn)$
 - ▶ Ordered lists: $O(m + n)$

Homework

1. Implement a self organizing list using linked list.
2. Implement a jump search with position estimation by interpolation for double-valued arrays.